M.Tech. (Online) Project Proposal and Work Plan

Student Name: **Monika Tyagi**

Student Number: **13-19-02-19-52-24-1-24515**

Month and Year of Enrolment into the Degree: **August 2024**

Stream: **DSBA**

No. of Core Course Credits Completed: **12**

No. of Elective Course Credits Completed: **16**

1. **Project Title**: **AI-Driven Quality Assistant Agent for STLC Automation**

2. Problem Description

The Software Testing Life Cycle (STLC) consists of multiple interdependent phases—requirements analysis, test planning, test design, test execution, defect reporting, and release readiness evaluation. In most organizations, these stages are supported by **siloed tools, manual handoffs, and unstructured workflows**, causing the output of one stage to be inconsistently validated before becoming the input for the next. This fragmentation leads to **coverage gaps, redundant testing effort, conflicting interpretations of requirements, and late detection of critical defects**, all of which increase delivery time and reduce software quality.

The problem this project aims to solve is the **lack of an integrated, intelligent, end-to-end quality framework** that connects and validates all STLC stages. Current automation tools focus primarily on execution and script maintenance, while upstream activities (requirements understanding, scenario generation) and downstream activities (defect triage, readiness assessment) remain largely manual. This results in inconsistent test assets, subjective decision-making, and high cost of quality.

The project seeks to answer three central questions:

1. How can multiple AI agents collaborate to ensure consistent, verified transitions across all STLC phases?
2. What guardrails are necessary to maintain accuracy, reliability, and auditability of AI-generated testing artifacts?
3. To what extent can an AI-driven, multi-agent Quality Assistant improve effort reduction, coverage, and early defect detection compared to traditional methods?

This work aims to create a unified, intelligent, and traceable quality pipeline that addresses long-standing gaps in software testing.

3. Problem Motivation

Addressing this problem is increasingly important as modern software systems grow in complexity while release cycles continue to shrink. Organizations are expected to deliver high-quality software faster, yet traditional QA processes rely heavily on manual interpretation, test design, triage, and readiness assessment. These manual steps introduce variability, slow decision-making, and create bottlenecks that make it difficult for teams to keep pace with rapid development practices such as Agile and DevOps. As a result, critical defects are often identified late, test coverage remains inconsistent, and the overall cost of quality increases significantly.

Existing automation tools focus primarily on test execution, leaving upstream tasks like requirements understanding and downstream tasks like defect analysis largely unaddressed. This gap leads to fragmented workflows where insights are not shared across stages, and valuable information—such as defect patterns or requirement ambiguities—is not leveraged to improve quality earlier in the cycle.

Introducing an **AI-driven, multi-agent Quality Assistant** has the potential to transform this landscape. By automating complex reasoning tasks, generating consistent test artifacts, and providing continuous quality intelligence, the solution can reduce manual effort, improve accuracy, and accelerate defect discovery.

Beyond operational efficiency, this project offers a strategic contribution: establishing an integrated, intelligent QA framework that aligns with industry trends toward autonomous testing, data-driven quality decisions, and scalable test optimization.

4. Related Work and Novelty

This project introduces, for the first time within the organization, an agent-based, end-to-end Quality Assistant framework for STLC automation. Unlike existing solutions that automate individual tasks, the proposed architecture orchestrates multiple intelligent sub-agents so that the **output of one STLC phase becomes the verified input of the next**, ensuring continuity and eliminating fragmentation. This coordinated flow establishes continuous quality intelligence—something not previously designed, analyzed, or implemented in the current organizational context. The project also defines a technically feasible and academically scoped solution, enabling structured evaluation of multi-agent orchestration, cross-stage validation, and lifecycle-wide quality insights.

This level of STLC integration represents a **novel contribution**, moving beyond point solutions toward a holistic, intelligent QA ecosystem.
The system will be developed and validated using **public datasets and open-source applications**, followed by controlled evaluation using **anonymized organizational artifacts**. This phased, privacy-preserving approach strengthens feasibility while enabling rigorous assessment of a holistic, lifecycle-integrated intelligent QA ecosystem.

5. Proposed Solution Approach

The proposed solution is an **AI-driven, multi-agent Quality Assistant Framework** designed to automate, streamline, and intelligently coordinate all phases of the Software Testing Life Cycle (STLC). The approach integrates Large Language Models (LLMs), automation frameworks, and agent-orchestrated reasoning to ensure that each lifecycle stage produces validated, high-quality outputs that seamlessly flow into the next. The goal is to replace fragmented, tool-specific workflows with a unified, end-to-end intelligent QA pipeline.

Multi-Agent Architecture with Central Orchestration

The system is anchored by an **Orchestrator Agent**, which governs interactions between specialized sub-agents, enforces stage-to-stage validation, and maintains global traceability. The key agents include:

- **Requirements Analysis Agent** – Converts unstructured requirements into structured intents, acceptance criteria, and requirement-to-test mappings through LLM-based semantic reasoning. It ensures clarity, consistency, and downstream alignment.
- **Test Scenario & Data Generation Agent** – Synthesizes comprehensive test scenarios, detailed test cases, boundary conditions, and synthetic test data. This agent ensures breadth of coverage and maintains direct traceability to requirement intents.
- **Automated Script Authoring Agent** –It supports script healing and abstraction-based modeling to minimize maintenance overhead.
- **Execution & Monitoring Agent** – Coordinates test execution, monitors runtime behavior, detects flaky tests, and aggregates logs, telemetry, and execution diagnostics for further analysis.
- **Defect Intelligence Agent** – Automatically identifies, classifies, and summarizes defects using clustering and pattern analysis. It generates structured defect reports, identifies probable root causes, and feeds insights back into the Orchestrator Agent to refine future prioritization.
- **Release Readiness Agent** – Synthesizes quality signals such as coverage, defect density, severity distribution, test pass trends, and risk indicators into an objective readiness score and dashboard for decision makers.

Technical Methods and Tools

The solution will leverage LLMs for language understanding and transformation, rule-based and ML-based models for defect analysis, microservices for modular agent deployment, and event-driven messaging for agent coordination. Feedback loops will allow defect patterns and change impact to continuously refine test prioritization and scenario generation.

Development and Validation Strategy

Initial development will use **public datasets, open-source applications, and community issue trackers** to enable reproducible experimentation. Later stages will incorporate **anonymized internal artifacts**—requirements, test cases, and defect logs—ensuring safe and realistic evaluation within organizational constraints.

6. Evaluation Plan

The evaluation of the proposed Quality Assistant Agent will follow a **phased, measurable, and evidence-driven approach** to determine whether the system adequately addresses gaps in the Software Testing Life Cycle (STLC). The first phase involves using **publicly available open-source applications, requirements datasets, and issue trackers**, enabling controlled experimentation and repeatability without organizational constraints. This allows benchmarking the agents' performance in requirements structuring, test generation quality, script accuracy, defect categorization, and readiness scoring in a safe, transparent environment.

Quantitative metrics in this phase will include:

- **Test coverage improvement** (e.g., scenario diversity, requirement-to-test traceability)
- **Reduction in manual effort** (time saved in test design and triage)
- **Accuracy of AI-generated artifacts** (validated through human review)
- **Defect detection efficiency** (early detection rate, clustering accuracy)

In the second phase, once the system stabilizes, evaluation will extend to **anonymized organizational datasets**, including sanitized requirements, historical test cases, and defect logs. This enables comparing the agent-driven workflow against baseline STLC processes currently in use.

Implementation experiments will include **agent-level simulations**, **end-to-end lifecycle dry runs**, and **A/B comparisons** between traditional and AI-augmented testing flows. Success will be demonstrated through measurable improvements in coverage, consistency, defect discovery, and release readiness accuracy.

7. Feasibility and Risks

The project is **highly feasible** within the expected timeline because it leverages publicly available datasets, open-source applications, and existing AI/automation frameworks, enabling rapid prototyping without dependency on restricted organizational data. The modular multi-agent design also supports incremental development and testing. However, there are risks to consider: LLM-generated artifacts may require iterative refinement, and integrating multiple agents may introduce coordination or latency challenges. Technical risks include variability in AI output quality, limited availability of high-quality open datasets for certain STLC stages, and potential constraints on compute resources. These risks can be mitigated through phased validation, guardrails, and fallback rule-based logic.

8. Work Plan

- **Phase 1 Term - Foundations & Core Prototype**

Term: Jan-Apr, Year: 2026

Task	Outcomes/Deliverables	From Date	To Date
Requirement analysis & literature survey	Problem statement, survey summary	01-Feb-2026	05-Feb-2026
Architecture & data strategy	Architecture spec, data plan	06-Feb-2026	15-Feb-2026
Test Scenario & Data Generation Agent prototype	Setup basic infrastructure, create frontend, backend and agent repository with initial setup. Agent integration, model selection. E2E stitch (reqs → scenarios → scripts) + baseline run	16-Feb-2026	25-Apr-2026
Phase-1 review & adjustments	Phase-1 report; backlog for Phase-2	26-Apr-2026	30-Apr-2026

- **Phase 2 Summer Term - Intelligence, Orchestration & Evaluation**

Term: Summer(May-July), Year: 2026

Task	Outcomes/Deliverables	From Date	To Date
Agent development	Generate v1, Implement other agents(Test data generation agent, Test script automation agent, Change impact analyses agent, Test execution agent, Self-healing test scripts agent, Bug report generation agent, Test summary reporting agent, release readiness evaluation agent.	01-May-2026	20-Jun-2026
Defect Analyzer + Orchestrator	Integrate all agents, orchestrated flow.	21-Jun-2026	30-June-2026
Integration & pipeline hardening	End-to-end implementation for v1	01-July-2026	25-July-2026

Interim evaluation on public data	End-to-end demo v1,Evaluation report v1	26-July-2026	31-July-2026

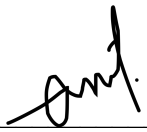
- **Phase 3 Term - Organizational Pilot & Finalization**

Term: Aug-Nov, Year: 2026

Task	Outcomes/Deliverables	From Date	To Date
Finalize frontend and backend Implementation and refinement	React web app and containerize frontend and backend services.	01-Aug-2026	31-Aug-2026
Deployment and evaluation with anonymized artifacts	Deployed application on cloud/server. Final metrics & analysis	01-Sep-2026	15-Oct-2026
Model tuning & documentation	Tuned models, user guide	16-Oct-2026	15-Nov-2026
Final report & defense	Dissertation & presentation	16-Nov-2026	30-Nov-2026

We approve the project proposal and workplan given above.





Signature of Internal Guide

Name: Kapil Madaan

Designation: Senior Engineering Manager

Organization: Alight Services India Pvt Ltd

Signature of IISc Faculty Mentor

Name: Punit Rathore

Designation: Assistant Professor

Department: Cyber Physical Systems